



PROGRAMMING WITH PYTHON

Lekha A

Computer Applications

PROGRAMMING WITH PYTHON

Python Basics and Standard Library Review of Data Types

Lekha A

Computer Applications



PROGRAMMING WITH PYTHON

Recap

- Introduction
- Advantages of Python
- Syntax and Semantics
- Program Translation
- Data Types



PROGRAMMING WITH PYTHON

Frozenset()

- Python has an inbuilt function called frozenset() that takes an iterable object as input and makes it immutable.
- Python Frozenset renders iterable objects unchangeable by freezing them.



PROGRAMMING WITH PYTHON

Frozenset()

- Frozenset is similar to set in Python, except that frozensets are immutable, which implies that once generated, elements from the frozenset cannot be added or removed.
- This function accepts any iterable object as input and transforms it into an immutable object.
- It is not assured that the order of the elements will be retained.



PROGRAMMING WITH PYTHON

Frozenset()

```
vowels = ('a', 'e', 'i', 'o', 'u')
fSet = frozenset(vowels)
print('The frozen set is:', fSet)
print('The empty frozen set is:', frozenset())
```

```
The frozen set is: frozenset({'u', 'o', 'e', 'a', 'i'})
The empty frozen set is: frozenset()
```

```
# frozensets are immutable
fSet.add('v')
```

```
-----
AttributeError      Traceback (most recent call last)
Cell In[3], line 2
      1 # frozensets are immutable
----> 2 fSet.add('v')

AttributeError: 'frozenset' object has no attribute 'add'
```



PROGRAMMING WITH PYTHON

Frozenset()

```
A = frozenset([1, 2, 3, 4])  
B = frozenset([3, 4, 5, 6])
```

```
# copying a frozenset  
C = A.copy()  
print(C)
```

```
frozenset({1, 2, 3, 4})
```

```
# union  
print(A.union(B))
```

```
frozenset({1, 2, 3, 4, 5, 6})
```

```
# intersection  
print(A.intersection(B))
```

```
frozenset({3, 4})
```

```
# difference  
print(A.difference(B))
```

```
frozenset({1, 2})
```

```
# symmetric_difference  
print(A.symmetric_difference(B))
```

```
frozenset({1, 2, 5, 6})
```



PROGRAMMING WITH PYTHON

Frozenset()

```
A = frozenset([1, 2, 3, 4])  
B = frozenset([3, 4, 5, 6])  
C = frozenset([5, 6])
```

```
# isdisjoint() method  
print(A.isdisjoint(C))
```

True

```
# issubset() method  
print(C.issubset(B))
```

True

```
# issuperset() method  
print(B.issuperset(C)) # Output: True
```

True

```
# Intersection
```

```
A&B
```

```
frozenset({3, 4})
```

```
# Union
```

```
A|C
```

```
frozenset({1, 2, 3, 4, 5, 6})
```

```
#Symmetric Difference
```

```
A^B
```

```
frozenset({1, 2, 5, 6})
```

```
#Set Difference
```

```
A-B
```

```
frozenset({1, 2})
```




PROGRAMMING WITH PYTHON

Frozenset()

```
A = frozenset([1, 2, 3, 4])  
B = frozenset([3, 4, 5, 6])  
C = frozenset([5, 6])
```

```
# isdisjoint() method  
print(A.isdisjoint(C))
```

True

```
# issubset() method  
print(C.issubset(B))
```

True

```
# issuperset() method  
print(B.issuperset(C)) # Output: True
```

True

```
# Intersection
```

```
A&B
```

```
frozenset({3, 4})
```

```
# Union
```

```
A|C
```

```
frozenset({1, 2, 3, 4, 5, 6})
```

```
#Symmetric Difference
```

```
A^B
```

```
frozenset({1, 2, 5, 6})
```

```
#Set Difference
```

```
A-B
```

```
frozenset({1, 2})
```



PROGRAMMING WITH PYTHON

Review of Data Types

- What is the output of the following code snippets?
 - `str = "ProgrammingwithPython" ; print (str[1:3])`
 - `var = "James" * 2 * 3; print(var)`
 - `x = 36 / 4 * (3 + 2) * 4 + 2; print(x)`
 - `v1, v2, v3 = 1, 2, "3"; print(v1 + v2 + v3)`
 - `x= 2 * 2 **3; print(x)`



PROGRAMMING WITH PYTHON

Review of Data Types

- What is the output of the following code snippets?
 - `str = "ProgrammingwithPython" ; print (str[1:3])` ro
 - `var = "James" * 2 * 3; print(var)` JamesJamesJamesJamesJamesJames
 - `x = 36 / 4 * (3 + 2) * 4 + 2; print(x)` 182.0
 - `v1, v2, v3 = 1, 2, "3"; print(v1 + v2 + v3)` Error
 - `x= 2 * 2 **3; print(x)` 16



PROGRAMMING WITH PYTHON

Review of Data Types

- What is the output of the following code snippets?
 - `tuple = (1, 2, 3); print(2 * tuple)`
 - `print(format("Python Programming", "^30"))`
 - `(2**2)**(2**3)`
 - `10<20 and 30<20 or 40 >10`
 - `print(0.1 + 0.2 == 0.3)`



PROGRAMMING WITH PYTHON

Review of Data Types

- What is the output of the following code snippets?
 - `tuple = (1, 2, 3); print(2 * tuple)` `(1, 2, 3, 1, 2, 3)`
 - `print(format("Python Programming", "^30"))` `Python Programming`
 - `(2**2)**(2**3)` `65536`
 - `10<20 and 30<20 or 40 >10` `True`
 - `print(0.1 + 0.2 == 0.3)` `False`



PROGRAMMING WITH PYTHON

Review of Data Structures

- `s = "foo"; t = "bar"; print("barf" in 2 * (s + t))`
- `s="ProgrammingwithPython"; print(s[:5])`
- `s="ProgrammingwithPython"; print(s[::-5])`
- `s="ProgrammingwithPython"; print(s[0]+s[-1])`
- `s="ProgrammingwithPython"; print(s[::-1][-1] + s[len(s)-1])`
- `s="ProgrammingwithPython"; print(s[::-1][::-5])`



PROGRAMMING WITH PYTHON

Review of Data Structures

- `s = "foo"; t = "bar"; print("barf" in 2 * (s + t))` True
- `s="ProgrammingwithPython"; print(s[::-5])` PagPn
- `s="ProgrammingwithPython"; print(s[::-5])` nPgaP
- `s="ProgrammingwithPython"; print(s[0]+s[-1])` Pn
- `s="ProgrammingwithPython"; print(s[::-1][-1] + s[len(s)-1])` Pn
- `s="ProgrammingwithPython"; print(s[::-1][::-5])` PagPn



PROGRAMMING WITH PYTHON

Review of Data Structures

- `print(

'$100 $200 $300'.count("$"),

'$100 $200 $300'.count("$", 5, 10),

'$100 $200 $300'.count("$", 5)

)`



PROGRAMMING WITH PYTHON

Review of Data Structures

- `print(' $100 $200 $300 '.count("$"),`
`' $100 $200 $300 '.count("$", 5, 10),`
`' $100 $200 $300 '.count("$", 5)`
`)`
3 1 2
counts occurrences of the given substring within the specified string.
The second and third parameters indicate start and end arguments. This method applies to the portion of the string beginning with the character position start and up to but not including end.



PROGRAMMING WITH PYTHON

Review of Data Structures

- `x = ["a", ["bb", ["ccc", "ddd"], "ee", "ff"], "g", ["hh", "ii"], "j"]`
- `print(x[1][1][0])`
- `print(x[4][0])`
- `print(x[1][2][1])`
- `print(x[2][0])`
- `print(x[3][1])`



PROGRAMMING WITH PYTHON

Review of Data Structures

- `x = ["a", ["bb", ["ccc", "ddd"], "ee", "ff"], "g", ["hh", "ii"], "j"]`
- `print(x[1][1][0])` `ccc`
- `print(x[4][0])` `j`
- `print(x[1][2][1])` `e`
- `print(x[2][0])` `g`
- `print(x[3][1])` `ii`



PROGRAMMING WITH PYTHON

Review of Data Structures

- `nameList = ['Harsh', 'Pratik', 'Bob', 'Dhruv'] ; print (nameList[1][-1])`
- `l = [1, 2, 3, 4]; l.append([5,6,7,8]); print(l)`
- `list1 = range(100, 110); print (list1.index(105))`
- `list1 = [1, 2, 3, 4, 5]; list2 = list1;`
- `list2[0] = 0; print( list1)`



PROGRAMMING WITH PYTHON

Review of Data Structures

- `nameList = ['Harsh', 'Pratik', 'Bob', 'Dhruv']; print (nameList[1][-1])`
- `l = [1, 2, 3, 4]; l.append([5,6,7,8]); print(l)` `[1, 2, 3, 4, [5, 6, 7, 8]]`
- `list1 = range(100, 110); print (list1.index(105))` `5`
- `list1 = [1, 2, 3, 4, 5]; list2 = list1;`
`[0, 2, 3, 4, 5]`
- `list2[0] = 0; print( list1)`



PROGRAMMING WITH PYTHON

Review of Data Structures

- `list1 = [2000, 2002] ; list2 = [2024, 2026] ;`
- `print ((list1 + list2)*2)`
- `list1 = ['physics', 'chemistry', 2000, 2024]`
- `print (list1[1][-1])`
- `list = [1, 2, 3, None, (1, 2, 3, 4, 5), ['Python', 'for', 'Programmers']]`
- `print(len(list))`



PROGRAMMING WITH PYTHON

Review of Data Structures

- `list1 = [2000, 2002] ; list2 = [2024, 2026]`
`[2000, 2002, 2024, 2026, 2000, 2002,`
`2024, 2026]`
- `print ((list1 + list2)*2)`
- `list1 = ['physics', 'chemistry', 2000, 2024]`
- `print (list1[1][-1])` y
- `list = [1, 2, 3, None, (1, 2, 3, 4, 5), ['Python', 'for', 'Programmers']]`
- `print(len(list))` 6



PROGRAMMING WITH PYTHON

Review of Data Structures

- `list = ['python', 'programming', '@', 'MCA', 'in', 'PESU']; print(list[0:6:2])`
- `list = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']; print(list[10:])`
- `list = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l']; print(list[10:])`
- `list = ['a', 'b', 'c'] * -3`



PROGRAMMING WITH PYTHON

Review of Data Structures

- `list = ['python', 'programming', '@', 'MCA', 'in', 'PESU']; print(list[0:6:2])`
`['python', '@', 'in']`
- `list = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']; print(list[10:])` `[]`
- `list = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l']; print(list[10:])` `['k', 'l']`
- `list = ['a', 'b', 'c'] * -3` `[]`



PROGRAMMING WITH PYTHON

Review of Data Structures

- `L1 = []; L1.append([1, [2, 3], 4])`
- `L1.extend([7, 8, 9]); print(L1[0][1][1] + L1[2])`
- `L1 = [1, 2, 3, 4]; L2 = L1; L3 = L1.copy(); L4 = L1`
- `L1[0] = [5]; print(L1, L2, L3, L4)`
- `L = [1, 3, 5, 7, 9]; print(L.pop(-3), end = ' ')`



PROGRAMMING WITH PYTHON

Review of Data Structures

- `L1 = []; L1.append([1, [2, 3], 4])`
- `L1.extend([7, 8, 9]); print(L1[0][1][1] + L1[2])`¹¹
- `L1 = [1, 2, 3, 4]; L2 = L1; L3 = L1.copy(); L4 = L1`
- `L1[0] = [5]; print(L1, L2, L3, L4)` `[[5], 2, 3, 4] [[5], 2, 3, 4] [1, 2, 3, 4] [[5], 2, 3, 4]`
- `L = [1, 3, 5, 7, 9]; print(L.pop(-3), end = ' ')`⁵



PROGRAMMING WITH PYTHON

Review of Data Structures

- `aList = [4, 8, 12, 16]; aList[1:4] = [20, 24, 28]; print(aList)`
- `sampleList = [10, 20, 30, 40]; del sampleList[0:6]; print(sampleList)`
- `aList = [10, 20, 30, 40, 50, 60, 70, 80]; print(aList[2:5])`
- `print(aList[:4])`
- `print(aList[3:])`



PROGRAMMING WITH PYTHON

Review of Data Structures

- `aList = [4, 8, 12, 16]; aList[1:4] = [20, 24, 28]; print(aList)` [4] 20, 24, 28
- `sampleList = [10, 20, 30, 40]; del sampleList[0:6]; print(sampleList)` []
- `aList = [10, 20, 30, 40, 50, 60, 70, 80]; print(aList[2:5])` [30, 40, 50]
- `print(aList[:4])` [10, 20, 30, 40]
- `print(aList[3:])` [40, 50, 60, 70, 80]



PROGRAMMING WITH PYTHON

Review of Data Structures

- `sampleList = [10, 20, 30, 40, 50]; sampleList.append(60); print(sampleList)`
- `sampleList.append(60); print(sampleList)`
- `aList = [5, 10, 15, 25]; print(aList[::-2])`
- `list1 = ['xyz', 'zara', 'Python']; print (max(list1))`
- `l = [None] * 10; print(len(l))`



PROGRAMMING WITH PYTHON

Review of Data Structures

- `sampleList = [10, 20, 30, 40, 50]; sampleList.append(60); print(sampleList)`
[10, 20, 30, 40, 50, 60]
- `sampleList.append(60); print(sampleList)`
[10, 20, 30, 40, 50, 60, 60]
- `aList = [5, 10, 15, 25]; print(aList[::-2])`
[25, 10]
- `list1 = ['xyz', 'zara', 'Python']; print (max(list1))`
zara
- `l = [None] * 10; print(len(l))`
10



PROGRAMMING WITH PYTHON

Review of Data Structures

- `print({1, 2, 3, 4, 5} - {3, 4} ^ {5, 6, 7})`
- `x = {1, 2, 3}; y = {1, 2}; y.issubset(x)`
- `s = {'foo', 'bar', 'baz', 'qux'}; s -= {'bar'}; print(s)`
- `s = {'foo', 'bar', 'baz', 'qux'}; s &= {'foo', 'baz', 'qux'}; print(s)`
- `s = {'foo', 'bar', 'baz', 'qux'}; s.discard('bar'); print(s)`



PROGRAMMING WITH PYTHON

Review of Data Structures

- `print({1, 2, 3, 4, 5} - {3, 4} ^ {5, 6, 7})` {1, 2, 6, 7}
- `x = {1, 2, 3}; y = {1, 2}; y.ispropersubset(x)` Error
- `s = {'foo', 'bar', 'baz', 'qux'}; s -= {'bar'}; print(s)` {'foo', 'baz', 'qux'}
- `s = {'foo', 'bar', 'baz', 'qux'}; s &= {'foo', 'baz', 'qux'}; print(s)` {'foo', 'baz', 'qux'}
- `s = {'foo', 'bar', 'baz', 'qux'}; s.discard('bar'); print(s)` {'baz', 'foo', 'qux'}



PROGRAMMING WITH PYTHON

Review of Data Structures

- `a = {'python' : 1, 'programs' : 2}; b = {'python' : 2, 'programs' : 1}; print (a == b)`
- `dictionary1 = {'GoogleDocs' : 1, 'Google' : 2, 'googledocs' : 3};`
`print(dictionary1['googledocs']);`
- `d1 = {"john":40, "peter":45}; d2 = {"john":466, "peter":45}; print (d1 > d2)`
- `test = {1:'A', 2:'B', 3:'C'}; del test[1]; test[1] = 'D'; del test[2]; print(len(test))`
- `a = {}; a['a']= 1; a['b']=[2, 3, 4]; print(a)`



PROGRAMMING WITH PYTHON

Review of Data Structures

- `a = {'python' : 1, 'programs' : 2}; b = {'python' : 2, 'programs' : 1}; print (a == b)` False
- `dictionary1 = {'GoogleDocs' : 1, 'Google' : 2, 'googledocs' : 3};`
`print(dictionary1['googledocs']);` 3
- `d1 = {"john":40, "peter":45}; d2 = {"john":466, "peter":45}; print (d1 > d2)` Error
- `test = {1:'A', 2:'B', 3:'C'}; del test[1]; test[1] = 'D'; del test[2]; print(len(test))` 2
- `a = {}; a['a']= 1; a['b']=[2, 3, 4]; print(a)` {'a': 1, 'b': [2, 3, 4]}



PROGRAMMING WITH PYTHON

Review of Data Structures

- `a = {'python' : 1, 'programs' : 2}; b = {'python' : 2, 'programs' : 1}; print (a == b)` False
- `dictionary1 = {'GoogleDocs' : 1, 'Google' : 2, 'googledocs' : 3};`
`print(dictionary1['googledocs']);` 3
- `d1 = {"john":40, "peter":45}; d2 = {"john":466, "peter":45}; print (d1 > d2)` Error
- `test = {1:'A', 2:'B', 3:'C'}; del test[1]; test[1] = 'D'; del test[2]; print(len(test))` 2
- `a = {}; a['a']= 1; a['b']=[2, 3, 4]; print(a)` {'a': 1, 'b': [2, 3, 4]}



PROGRAMMING WITH PYTHON

Recap

- Review of Data Types



THANK YOU

Lekha A

Computer Applications